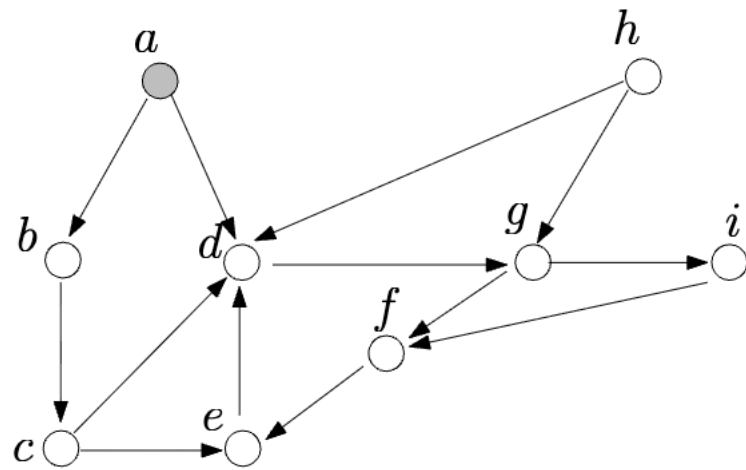


Graph Traversal

Depth First Search (Stack-Based)

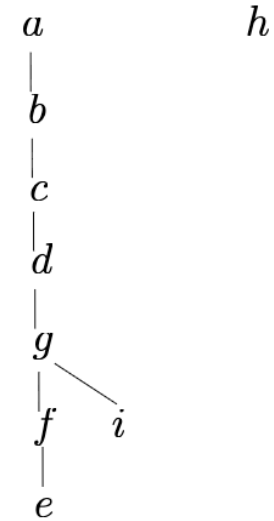
- Use a stack S to manage the vertices we visited
- At the beginning, color all vertices in the graph white and create an empty DFS tree T .
- Initially, S contains an arbitrary node v as the start node, and color it gray (which means “in the stack”). Make v the root of T .
- Repeat the following until S is empty:
 - Let v be the vertex at the top of the Stack S
 - Does v still have white out-neighbor?
 - If so: let it be u ; Push u into S , color it gray, and set u as a child of v in T
 - Otherwise, pop v from S and color it black (meaning v is done).
- If there are still white vertices, repeat the above by restarting from an arbitrary white vertex v' , creating a new DFS-tree rooted at v' .

Depth First Search (Stack-Based)



DFS tree
a

DFS forest



We will create a DFS-forest, which consists of 2 DFS-trees in this example:

Time complexity $O(m + n)$, with an adjacency list

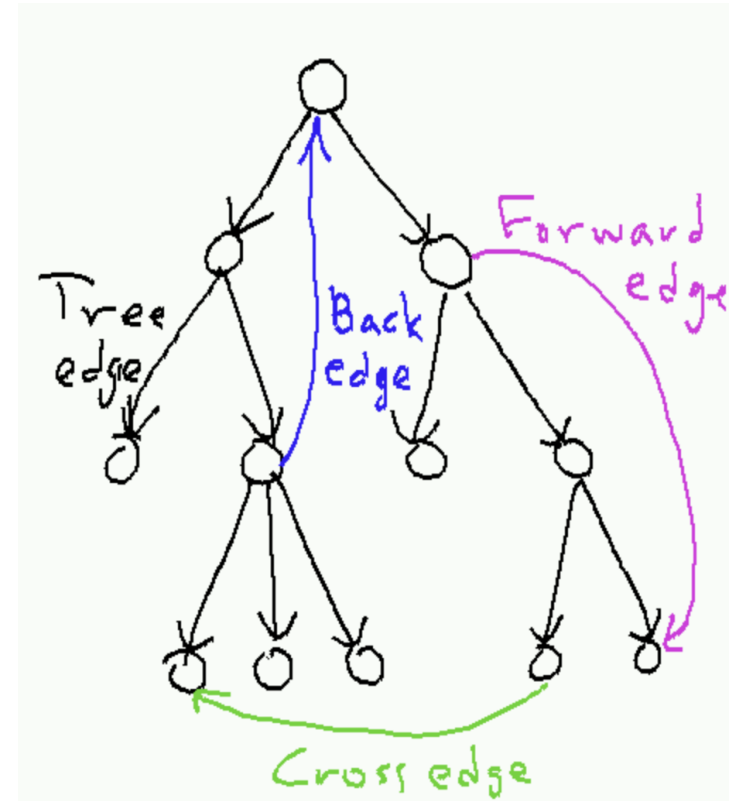
- For every vertex v , remember which is the next out-neighbor to explore.
- Use an array to remember the colors of all vertices
- $O(m + n)$ stack operations

The Ancestor-Descendant Property:

- u is an ancestor of v in the DFS-forest if and only if the following holds: u is already in the stack when v enters the stack.

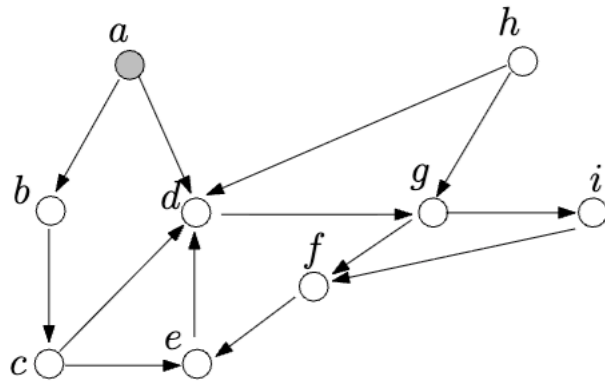
Each edge (u, v) of G can be classified into:

- **Forward edge**: if u is a proper ancestor of v in a DFS-tree
- **Back edge**: if u is a descendant of v in a DFS-tree
- **Cross edge**: if neither the above applies

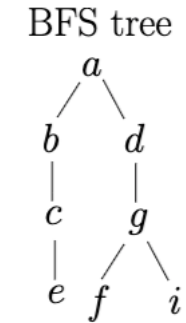


- BFS: starts traversing the graph from root node and explores all the neighbours. Then, it selects the nearest node and explore all the unexplored nodes. It follows the same process for each of the nearest node until it finds the goal.
 - Step 1: SET color = white (STATUS = 1) for each node in G
 - Step 2: **Enqueue** the starting node A and set its color to gray (STATUS = 2, “in the queue”)
 - Step 3: Repeat Steps 4 and 5 until **QUEUE** is empty
 - Step 4: **Dequeue** a node N. Process it and set its color to black (STATUS = 3 processed).
 - Step 5: **Enqueue** all white neighbors of N and color them gray (STATUS = 2)
 - Make all these neighbors as the child nodes of N in the BFS tree T
 - Step 6: EXIT

Breadth First Search



BFS tree
a



- Given two vertices $u, v \in V$, a shortest path from u to v is a path of the minimum length from u to v .
- Assume that each edge has a length of 1 unit.
- The shortest path from the start vertex A to any vertex, say, x is simply the path from A to node x in the BFS tree!